

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

Name: Challa Priyadarshini

Reg No. : 192311118

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving

1. RSA Encryption Analysis

Given:

- $p = 11$
- $q = 13$
- Message $M = 9$
- Public exponent $e = 7$

a. Compute n and $\phi(n)$

Step 1: Calculate n

$$\begin{aligned} n &= p \times q \\ n &= 11 \times 13 = \boxed{143} \end{aligned}$$

Step 2: Calculate Euler's Totient

$$\begin{aligned} \phi(n) &= (p - 1)(q - 1) \\ \phi(143) &= (11 - 1)(13 - 1) \\ &= 10 \times 12 = \boxed{120} \end{aligned}$$

So:

- $n = 143$
- $\phi(n) = 120$

b. Choose $e = 7$ and find private key d

RSA requires:

$$e \times d \equiv 1 \pmod{\phi(n)}$$

Therefore:

$$7d \equiv 1 \pmod{120}$$

We need the modular inverse of 7 modulo 120.

$$\begin{aligned} 7 \times 103 &= 721 \\ 721 &= 6(120) + 1 \end{aligned}$$

Therefore:

$$\boxed{d = 103}$$

Keys:

- Public key = $\boxed{(e, n) = (7, 143)}$
- Private key = $\boxed{(d, n) = (103, 143)}$

c. Encrypt message $M = 9$

RSA encryption formula:

Substitute:

$$C = 9^7 \text{ } 143$$

Calculate:

$$\begin{aligned} 9^2 &= 81 \\ 9^4 &= 81^2 = 6561 \text{ } 143 = 126 \end{aligned}$$

$$\begin{aligned}
 9^7 &= 9^4 \times 9^2 \times 9 \\
 &= 126 \times 81 \times 9 \text{ } 143 \\
 \boxed{C} &= 48
 \end{aligned}$$

So, the **ciphertext** is **48**.

d. Decrypt and verify

RSA decryption formula:

$$M = C^d \text{ } n$$

Substitute:

$$\begin{aligned}
 M &= 48^{103} \text{ } 143 \\
 \boxed{M} &= 9
 \end{aligned}$$

The decrypted message is the same as the original message:

$$\boxed{9 = 9}$$

Therefore, **RSA encryption and decryption are verified successfully.**

Summary

Parameter	Value
p	11
q	13
n	143
$\phi(n)$	120
e	7
d	103
Original M	9
Ciphertext C	48
Decrypted M	9

e. How changing e affects security

The value e is the **public encryption exponent**.

- e must satisfy:

$$1 < e < \phi(n) \\ \text{and} \\ \gcd(e, \phi(n)) = 1$$

- A different valid e produces a different private key d .
- In real RSA, commonly used $e = 65537$ provides efficient encryption while satisfying the required mathematical conditions.
- Changing e alone does not make small RSA secure. Security mainly depends on using very large, properly generated prime numbers p and q .
- In this example, $p = 11$ and $q = 13$ are extremely small, so the RSA system can be broken easily by factoring $n = 143$.

Question 2:

Block Cipher Mode Evaluation

Given:

- Plaintext blocks:

$$P = [1010, 1010, 1111, 1010]$$

- Key:

$$K = 1100$$

- IV:

$$IV = 0000$$

Assumption: For this simple example, encryption is performed using XOR:

$$E(X) = X \oplus K$$

a. Encryption using ECB and CBC

1. ECB Mode

In ECB, each plaintext block is encrypted **independently**.

$$C_i = P_i \oplus K$$

Block	Plaintext	Key	Ciphertext
P_1	1010	1100	0110
P_2	1010	1100	0110
P_3	1111	1100	0011
P_4	1010	1100	0110

Therefore:

$$C_{ECB} = [0110, 0110, 0011, 0110]$$

Notice that the repeated plaintext block **1010** produces the same ciphertext **0110**.

2. CBC Mode

CBC uses the previous ciphertext block.

Formula:

$$C_i = E(P_i \oplus C_{i-1})$$

where:

$$C_0 = IV = 0000$$

Block 1

$$P_1 \oplus IV = 1010 \oplus 0000 = 1010$$

$$C_1 = 1010 \oplus 1100 = \boxed{0110}$$

Block 2

$$P_2 \oplus C_1 = 1010 \oplus 0110 = 1100$$

$$C_2 = 1100 \oplus 1100 = \boxed{0000}$$

Block 3

$$P_3 \oplus C_2 = 1111 \oplus 0000 = 1111$$

$$C_3 = 1111 \oplus 1100 = \boxed{0011}$$

Block 4

$$P_4 \oplus C_3 = 1010 \oplus 0011 = 1001$$

$$C_4 = 1001 \oplus 1100 = \boxed{0101}$$

Therefore:

$$C_{CBC} = [0110, 0000, 0011, 0101]$$

b. Compare the ciphertext outputs

Mode	Ciphertext
ECB	[0110, 0110, 0011, 0110]
CBC	[0110, 0000, 0011, 0101]

The repeated plaintext blocks are:

$$P_1 = P_2 = P_4 = 1010$$

In **ECB**, they produce the same ciphertext:

$$1010 \rightarrow 0110$$

In **CBC**, they produce different ciphertexts:

$$1010 \rightarrow 0110, 0000, 0101$$

c. Which mode hides patterns better?

$$\boxed{\text{CBC hides patterns better than ECB}}$$

Reason:

- **ECB:** Each block is encrypted independently.
- Therefore, identical plaintext blocks produce identical ciphertext blocks.
- This can reveal patterns in the plaintext.
- **CBC:** Each plaintext block is combined with the previous ciphertext block before encryption.
- Therefore, even identical plaintext blocks can produce different ciphertext blocks.

Question 3:

DES vs 3-DES vs Blowfish Performance Study

Given:

- **DES:** Key = 56 bits, time = 2 ms/block
- **3-DES:** Key = 168 bits, time = 6 ms/block
- **Blowfish:** Key = 128 bits, time = 1 ms/block
- Number of blocks = **1000**

1. Calculate total encryption time

Formula:

$$\text{Total Time} = \text{Time per block} \times \text{Number of blocks}$$

Algorithm	Key Size	Time/Block	Total Time for 1000 Blocks
DES	56 bits	2 ms	$2 \times 1000 = 2000 \text{ ms} = 2 \text{ s}$
3-DES	168 bits	6 ms	$6 \times 1000 = 6000 \text{ ms} = 6 \text{ s}$
Blowfish	128 bits	1 ms	$1 \times 1000 = 1000 \text{ ms} = 1 \text{ s}$

Therefore:

Blowfish = 1 second

DES = 2 seconds

3-DES = 6 seconds

- Encryption Time for 1000 Blocks
- Comparison of total encryption time based on the given time per block.

2. Trade-offs between security and performance

DES

- Smallest key: **56 bits**
- Faster than 3-DES but slower than Blowfish in this example.
- **Not secure for modern applications** because its key size is too small.
- Suitable mainly for historical/educational study.

3-DES

- Uses a much larger effective key space than DES and is considerably stronger than DES.
- However, it is **slowest**, taking 6 seconds for 1000 blocks.
- Its performance makes it unsuitable for many modern high-speed applications.

Blowfish

- Key size given: **128 bits**
- Fastest in this comparison.
- Provides a much better performance/security balance than DES and 3-DES for this simplified comparison.
- However, Blowfish is an older design and is generally not the first choice for a new system.

3. Best algorithm for secure real-time communication

Based strictly on the three algorithms and the given data:

Blowfish

Reason :

- Fastest: **1 second**
- Larger key size than DES: **128 bits**
- Much faster than 3-DES: **1 s vs 6 s**
- Good balance between **performance and security** among the three options.

Final Answer

Blowfish is the best choice among DES, 3-DES, and Blowfish for this question.

Question 4:

Diffie-Hellman Key Exchange Analysis

Given:

a. Prime number: $p = 23$ Generator: $g = 5$

b. User A private key: $a = 6$

c. User B private key: $b = 15$

d. Compute public keys

The public key formula is:

$$Public = g^{private} \bmod p$$

User A:

$$\begin{aligned} A &= 5^6 \bmod 23 \\ 5^6 &= 15625 \\ 15625 \bmod 23 &= \boxed{8} \end{aligned}$$

So, A's public key = 8

User B:

$$\begin{aligned} B &= 5^{15} \bmod 23 \\ 5^{15} \bmod 23 &= \boxed{19} \end{aligned}$$

So, B's public key = 19

e. Compute the shared secret key

User A calculates:

$$\begin{aligned} K_A &= B^a \bmod p \\ K_A &= 19^6 \bmod 23 \end{aligned}$$

$$\boxed{K_A = 2}$$

User B calculates:

$$\begin{aligned} K_B &= A^b \bmod p \\ K_B &= 8^{15} \bmod 23 \\ \boxed{K_B} &= 2 \end{aligned}$$

Therefore:

$$\boxed{\text{Shared Secret Key} = 2}$$

Both users obtain the same secret.

f. Man-in-the-Middle (MITM) attack

The basic Diffie-Hellman protocol **does not authenticate the participants**. An attacker, Eve, can intercept the public values.

Normal exchange:

$$\begin{aligned} A &\rightarrow B: 8 \\ B &\rightarrow A: 19 \end{aligned}$$

An attacker can replace these values:

$$A \rightarrow Eve: 8$$

Eve sends a fake public value to B:

$$Eve \rightarrow B: X$$

Similarly, Eve intercepts B's value and sends another fake value to A:

$$\begin{aligned} B &\rightarrow Eve: 19 \\ Eve &\rightarrow A: Y \end{aligned}$$

Now A and B do **not** actually share the same secret with each other. Instead, Eve establishes separate secrets with A and B.

Conceptually:

$$A \leftrightarrow Eve \leftrightarrow B$$

Eve can then potentially intercept and relay communications between them.

g. Secure method to prevent the attack

The main solution is **authentication**.

Use **authenticated Diffie-Hellman**, such as:

- **Digital signatures** to authenticate the exchanged public keys.
- **Digital certificates/PKI** to verify identities.
- Use a modern authenticated key-exchange protocol such as **TLS**, which combines key exchange with authentication.

Final Summary

Item	Result
p	23
g	5
A's private key	6
B's private key	15
A's public key	8
B's public key	19
Shared secret	2
Main vulnerability	Man-in-the-Middle attack
Prevention	Authentication + digital signatures/certificates

Question 5:

ECC vs RSA Comparison

Given:

- a. RSA key size = 1024 bits
- b. ECC key size = 160 bits
- c. Both are assumed to provide equivalent security.

➤ Device = IoT device with limited processing capacity

d. Computational Cost

RSA:

- Uses a larger **1024-bit key**.
- Requires more computation for key generation and private-key operations.
- Needs more processing power.

ECC:

- Uses a much smaller **160-bit key**.
- Requires less computation for many public-key operations.
- Suitable for devices with limited processing capacity.

Therefore:

ECC has lower computational requirements for this scenario

e. Power and Memory Consumption

Memory:

ECC uses a smaller key:

$$160 < 1024$$

So, ECC requires less memory to store keys and related cryptographic data.

ECC uses less memory

Power:

Because ECC generally requires less computational work for comparable security, it can use less energy.

ECC generally consumes less power

This is especially useful for battery-powered IoT devices.

f. Which is better for IoT?

ECC is better for the given IoT scenario
--

Reasons:

1. ECC uses a **smaller key size**.
2. It requires **less memory**.
3. It generally requires **less processing power**.
4. It can reduce **power consumption**.
5. Smaller cryptographic data reduces **communication overhead**.
6. It is well suited for **resource-constrained IoT devices**.